Summary

For quick reference, the following table offers a thorough analysis of the distinctive characteristics, applicability, examples, benefits, and drawbacks of each creational design pattern.

Pattern Name	Distinctive Feature	Applicability	An Example	Pros	Cons
Singleton	Ensures only one instance of a class exists.	When a single instance is required to handle several processes, such as configuration management or resource pooling.	Managing a single database connection pool.	- Global access to instance - Ensures a single instance is initialized.	- Can hinder unit testing - Can introduce hidden dependencies.
Factory Method	Defines an object creation interface where the type is decided by the subclasses.	When you want to assign object creation to subclasses for more flexibility.	Creating different document types (PDF, Word, etc.) using a common Document class.	- Supports open-closed principle - Decouples creator and products.	- Increases the number of classes - Subclasses must be implemented.
Abstract Factory	Provides a way to create groups of related objects without defining concrete classes.	When you want to abstract object creation, you need to make sure that newly formed objects are compatible within a family.	Creating GUI elements in different OS themes (Windows, macOS) using related factories.	- Ensures product compatibility - Supports open-closed principle.	- Can be complex to implement - Extending with new products can be challenging.
Builder	Separates construction of complex	When you need to create	Building a custom meal	- Allows step-by-step construction	- Can be verbose for simple objects

Pattern Name	Distinctive Feature	Applicability	An Example	Pros	Cons
	objects from their representation.	complex objects step by step and control the process.	at a fast-food restaurant.	- Supports different representations.	- Requires a Director (optional).
Prototype	Creates new objects by copying an existing object.	When creating objects is more complex than copying existing ones or objects have similar structures.	Cloning a complex object, like a customized car configuration.	- Simplifies object creation - Reduces subclassing.	- Deep copying can be complex - Some languages lack built-in support for cloning.

← Previous

Prototype Pattern

Next →

Introduction

Mark as Completed